Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет информационных технологий и управления
Кафедра интеллектуальных информационных технологий

ОТЧЁТ

по лабораторной работе № 1
по дисциплине

ЛОГИЧЕСКИЕ ОСНОВЫ ИНТЕЛЛЕКТУАЛЬНЫХ СИСТЕМ

Студенты гр. 421702:

Сайковский А. В.
Мяло В. В.
Вашкевич М. В.

Проверил:

В. П. Ивашенко

Минск 2026

ХОД ВЫПОЛНЕНИЯ ЛАБОРАТОРНОЙ РАБОТЫ

Вариант 1

Тема: Программирование операций обработки и преобразования формул прикладных и неклассических логик.
Цель: Приобрести навыки программирования алгоритмов обработки данных в неклассических логических моделях решения задач.
Задание: Реализовать прямой нечёткий логический вывод с использованием треугольной нормы min и нечёткой импликации Гёделя.

Описание работы

Разработана программа на языке Python, выполняющая прямой нечёткий логический вывод по фактам и правилам из текстового файла. Факты задаются конечными нечёткими множествами, а правила имеют форму `A(x) ~> B(y)` (в файле фактически используется слитная запись `~>`). Для каждого применимого правила строится нечёткое бинарное отношение по импликации Гёделя, после чего вычисляется max-min композиция с входным фактом. Новые факты добавляются в базу до прекращения её расширения.

При реализации использованы стандартные средства Python, модули `re` и `pathlib`, списки, кортежи, множества и словари. Совместимость фактов определяется совпадением наборов элементов их универсумов. Уже известные результаты обнаруживаются сравнением списков пар, поэтому при таком сравнении учитывается порядок элементов. Предполагается, что входные степени принадлежности находятся в диапазоне $[0, 1]$, а элементы внутри множества не повторяютсяю.

Теоретические сведения

Нечёткое множество A на универсуме X определяется функцией принадлежности $\mu_A : X \to [0, 1]$. Значение $\mu_A(x)$ задаёт степень принадлежности элемента x множеству A. Для конечного универсума множество записывается в виде

$$A = \{\langle x_1, \mu_A(x_1)\rangle, \ldots, \langle x_n, \mu_A(x_n)\rangle\}. \tag{1}$$

Нечёткий предикат — предикат, истинностное значение которого принадлежит $[0, 1]$. Нормальным называется нечёткое множество высоты $h(A) =$

1, где $h(A) = \sup_x \mu_A(x)$. Если $h(A) < 1$, множество является субнормальным.

Треугольная норма (t-норма) — бинарная операция на $[0, 1]$, обобщающая нечеткую конъюнкцию. В данной работе используется

$$T(a, b) = \min(a, b). \tag{2}$$

Импликация Гёделя задаётся выражением

$$I_G(a, b) = \begin{cases} 1, & a \le b, \\ b, & a > b. \end{cases} \tag{3}$$

Нечёткое бинарное отношение, соответствующее правилу $A(x) \to B(y)$, определяется на $X \times Y$ следующим образом:

$$R(x, y) = I_G\big(\mu_A(x), \mu_B(y)\big). \tag{4}$$

Обобщённый modus ponens позволяет по входному факту $A'(x)$ и правилу $A(x) \to B(y)$ получить новое заключение $B'(y)$. Используется max-min композиция, которая на конечном универсуме принимает вид

$$\boxed{\mu_{B'}(y) = \max_{x \in X} \min\big(\mu_{A'}(x), R(x, y)\big).} \tag{5}$$

Прямой нечёткий вывод — последовательное получение новых фактов из уже имеющихся фактов и нечётких правил. Результат каждого шага может использоваться в последующих шагах. Обход правил повторяется, пока количество различных фактов в базе не перестанет увеличиваться.

Формат базы знаний

Входные данные располагаются в текстовом файле `input.txt`, который считывается функцией `main()` программы `lab1.py`. Каждое определение нечёткого множества и каждое правило записывается на отдельной строке. Определения задаются именем и списком пар `<элемент, степень>`; ниже записываются правила со знаком `~>`. Допускаются пустые строки. Следующая упрощённая грамматика описывает основные конструкции формата:

```
<база знаний>::= <список фактов>|<список правил><новая строка><список правил>
<список фактов>::= <факт>|<факт><новая строка><список фактов>
<список правил>::= <правило>|<правило><новая строка><список правил>
<факт>::= <имя нечёткого предиката><равенство><нечёткое множество>
<правило>::= <имя нечёткого предиката><нечеткая импликация><имя нечёткого
 ↪  предиката>.
<нечёткое множество>::=<открывающаяся фигурная скобка><список пар нечёткой
 ↪  принадлежности><закрывающаяся фигурная скобка>
<список пар нечёткой принадлежности>::=<пара нечёткой принадлежности>|<пара
 ↪  нечёткой принадлежности><запятая><список пар нечёткой принадлежности>
<пара нечёткой принадлежности>::=<открывающаяся угловая
 ↪  скобка><элемент><запятая><степень принадлежности><закрывающаяся угловая
 ↪  скобка>
<имя нечеткого предиката>::= <имя><открывающаяся полукруглая
 ↪  скобка><имя><закрывающаяся полукруглая скобка>
<элемент>::=<имя>
<имя>::=<буква>|<буква><символы>
<символы>::=<символ>|<символ><символы>
<степень принадлежности>::=<действительное число с 0 по 1>
<символы>::=<буква>|<цифра>
<множество>::=<ориентированное множество> |
<неориентированное множество>
<неориентированное множество>::={ <список элементов> }
<ориентированное множество>::=(<элемент>,<список элементов>)
<список элементов>::=<элемент> ,<элемент> ,<элемент>
<имя нечёткого множества>::=<имя>
<действительное число с 0 по
 ↪  1>::=<единица>|<единица><точка><нули>|<действительное число с 0 до 1>
<действительное число с 0 до 1>::=<ноль>|<ноль><точка><цифры>
<нули>::=<ноль>|<ноль><нули>
<цифра>::=<цифра>|<цифра><цифры>
<символ>::=<буква>|<цифра>
<цифра>::=0|...|9
```

```
<буква>::=A|...|z
<единица>::= 1
<ноль>::= 0
<точка>::= .
<запятая>::= ,
<равенство>::= =
<нечеткая импликация>::= ~>
<открывающаяся угловая скобка>::= <
<закрывающаяся угловая скобка>::= >
<открывающаяся фигурная скобка>::= {
<закрывающаяся фигурная скобка>::= }
<открывающаяся полукруглая скобка>::= (
<закрывающаяся полукруглая скобка>::= )
```

Содержимое файла `input_original.txt`:

```
A = {<x1, 0.0>, <x2, 0.1>, <x3, 0.3>, <x4, 1.0>}
B = {<y1, 1.0>, <y2, 0.8>, <y3, 0.2>, <y4, 0.0>, <y5, 0.0>}
C = {<x1, 0.1>, <x2, 1.0>, <x3, 0.3>, <x4, 0.0>}

A(x) ~> B(y)
C(x) ~> A(y)
```

Структуры данных и функции программы

Словарь `facts` сопоставляет имени нечёткого множества список пар `(element, degree)`. Список `rules` хранит пары имён посылки и заключения. Бинарное нечёткое отношение представляется списком троек `(x, y, implication_degree)`.

- `read_knowledge_base(filename)` — считывает факты и правила с помощью регулярных выражений и преобразует степени принадлежности в числа;
- `build_godel_relation(antecedent, consequent)` — строит отношение по формуле (3);
- `max_min_composition(observation, relation)` — вычисляет заключение по формуле (5);
- `apply_fuzzy_rule(...)` — выполняет один шаг нечёткого вывода;
- `find_compatible_facts(...)` — находит факты, имеющие тот же набор элементов, что и посылка;

- `format_fuzzy_set(...)` — создаёт текстовую запись множества для вывода;
- `forward_fuzzy_inference(...)` — организует полный прямой вывод до отсутствия новых фактов;
- `main()` — запускает считывание и выводит результаты.

Схемы алгоритмов программы

Блок-схемы из архива исходных схем иллюстрируют основные этапы алгоритма. В текущей реализации проверка наличия полученного множества выполняется в `forward_fuzzy_inference()` сравнением списков пар. Считывание базы и форматирование результатов описаны текстом.

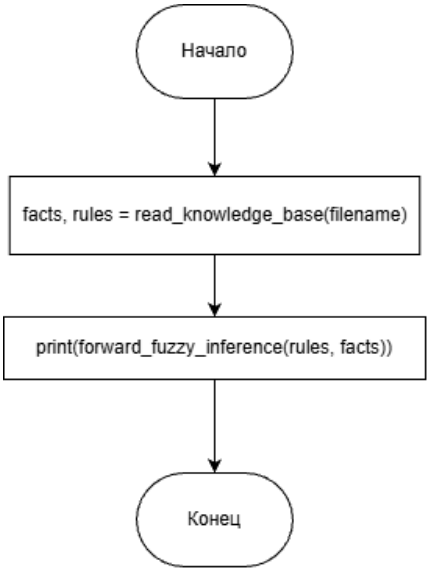


Рис. 1 Схема функции `main()`

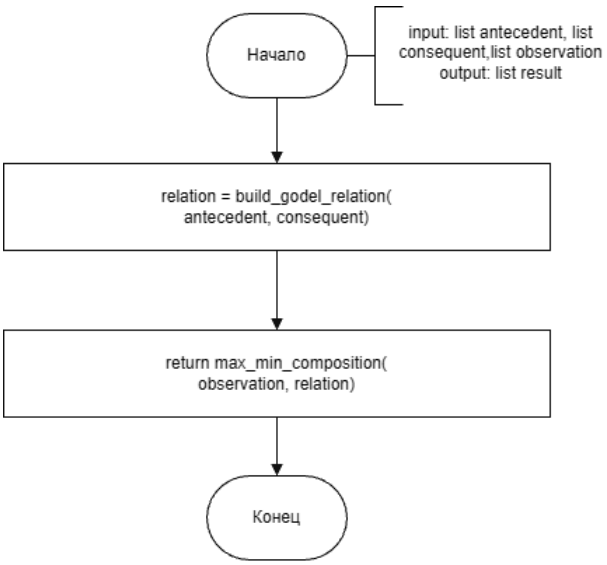


Рис. 2 Схема функции `apply_fuzzy_rule()`

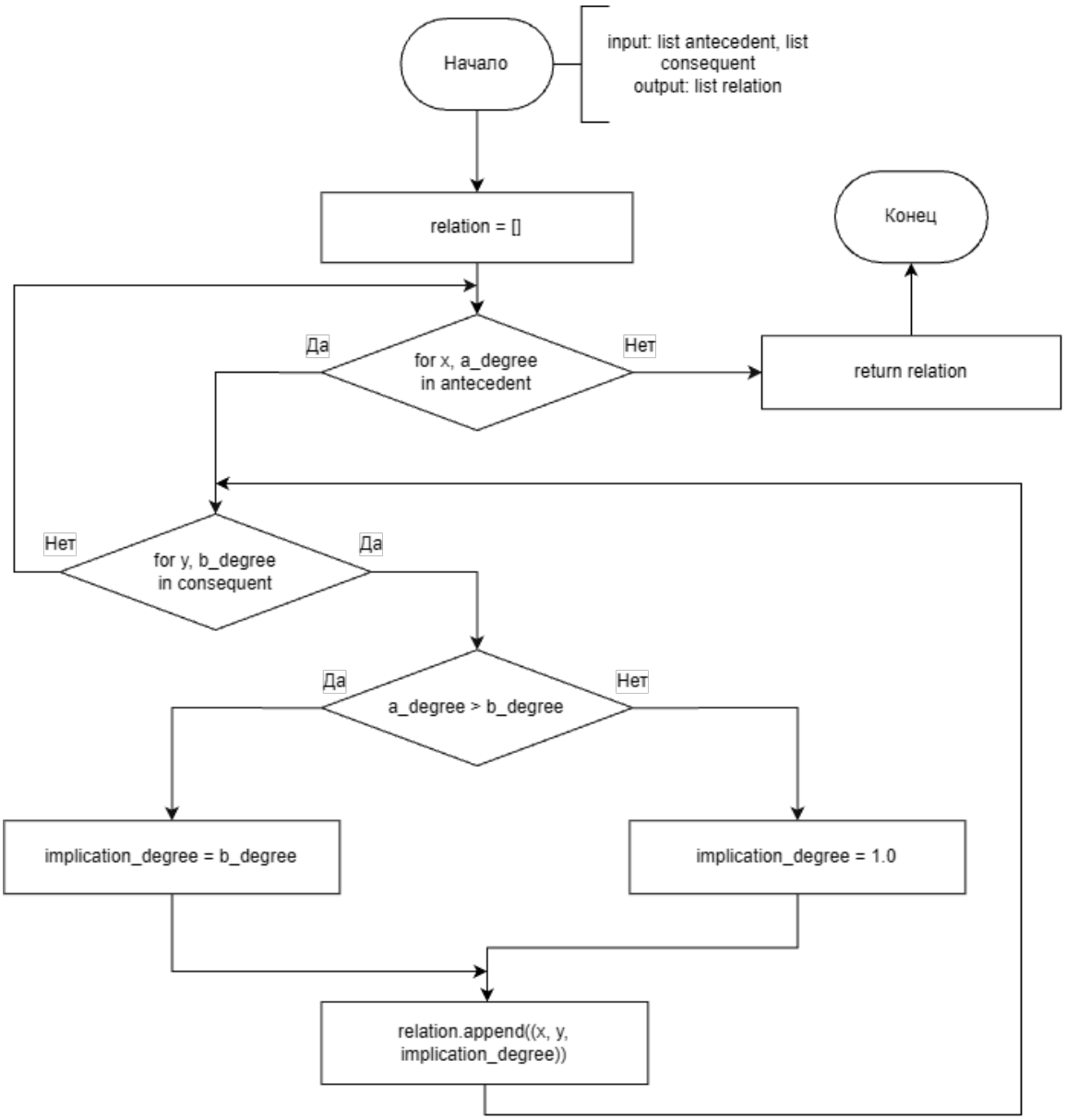


Рис. 3 Схема функции `build_godel_relation()`



Рис. 4 Схема функции `max_min_composition()`

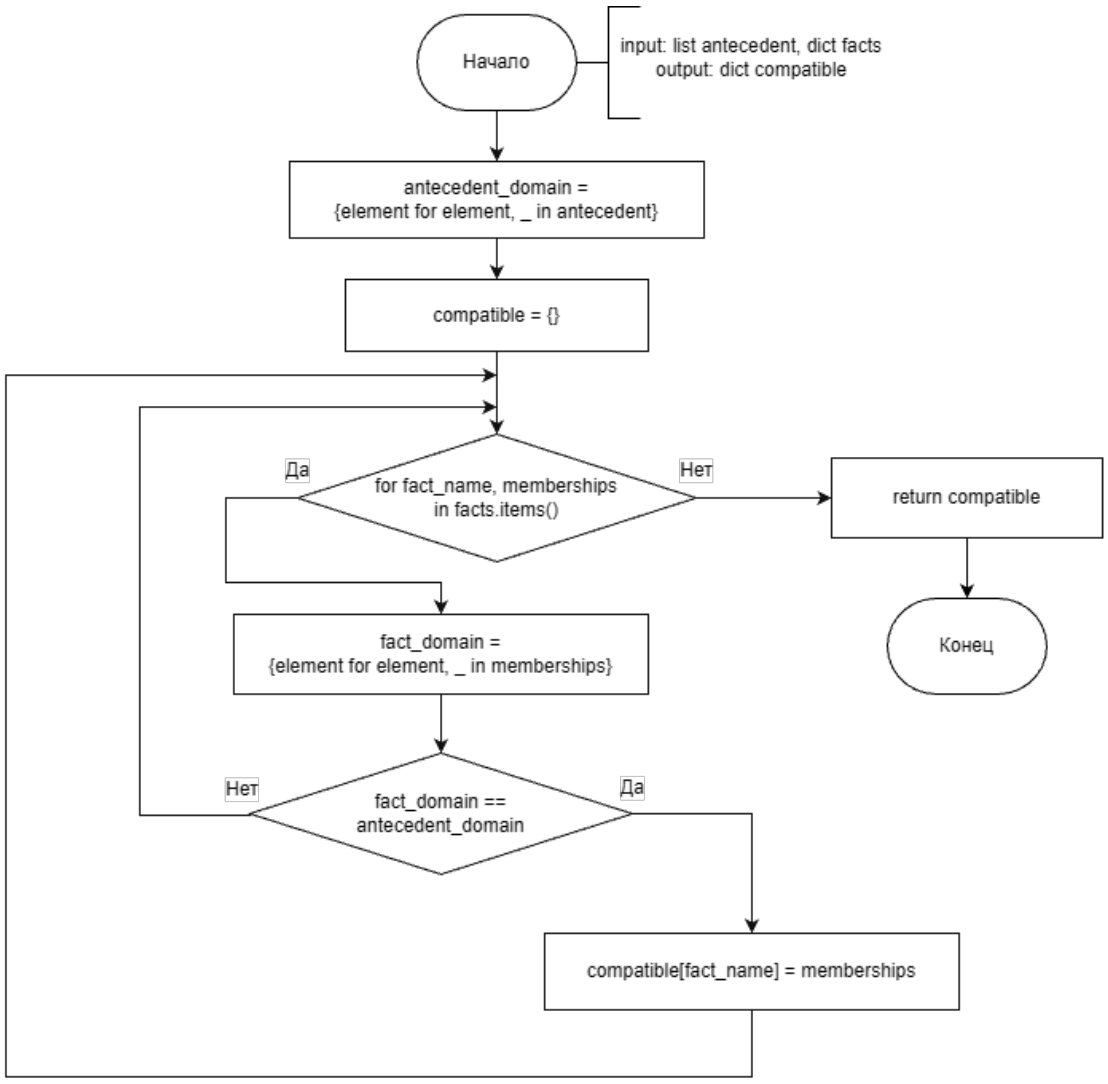


Рис. 5 Схема функции `find_compatible_facts()`



Рис. 6 Схема функции `forward_fuzzy_inference()`

Демонстрация результатов работы программы

Программа `lab1.py` запущена с базой знаний из файла `input_original.txt`, приведённой выше.

Полный результат запуска:

```
{A, A(x)~>B(y)} |~ _1 = {<y1, 1.0>,<y2, 0.8>,<y3, 0.2>,<y4, 0.0>,<y5, 0.0>}
{C, A(x)~>B(y)} |~ _2 = {<y1, 1.0>,<y2, 1.0>,<y3, 1.0>,<y4, 0.1>,<y5, 0.1>}
{A, C(x)~>A(y)} |~ _3 = {<x1, 1.0>,<x2, 1.0>,<x3, 1.0>,<x4, 1.0>}
{C, C(x)~>A(y)} |~ _4 = {<x1, 0.0>,<x2, 0.1>,<x3, 0.3>,<x4, 1.0>}
{_3, A(x)~>B(y)} |~ _5 = {<y1, 1.0>,<y2, 1.0>,<y3, 1.0>,<y4, 1.0>,<y5, 1.0>}
```

Личный вклад

Сайковский А. В.: реализация max-min композиции и механизма прямого нечёткого вывода.

Мяло В. В.: реализация импликации Гёделя.

Вашкевич М. В.: реализация работы с регулярными выражениями и механизмов обработки строк.

Вывод

В ходе лабораторной работы изучен и программно реализован прямой нечёткий логический вывод с использованием импликации Гёделя и треугольной нормы минимума. Освоены представление нечётких множеств функциями принадлежности, построение отношения для правила и получение заключения с помощью max-min композиции. Реализованы считывание базы знаний, подбор совместимых наблюдений и последовательное применение правил до прекращения расширения базы фактов.

На базе знаний из файла `input_original.txt` получены 10 строк результатов, из которых 7 соответствуют новым различным фактам. Остальные три воспроизводят исходные множества и повторно в базу не добавляются. Полученные факты используются в последующих шагах, что демонстрирует выполнение цепочек вывода, а проверка уже известных результатов позволяет завершить обработку циклически связанных правил для данного набора данных.

При анализе свойств вывода установлено, что высота заключения не превосходит высоту наблюдения, а при нормальном заключении правила эти

высоты совпадают. Для наблюдения $A' = A$ выведено равенство $\mu_{B'}(y) = \min(h(A), \mu_B(y))$, объясняющее условия точного воспроизведения B. Таким образом, нормальность результата и совпадение с заключением правила определяются не только видом импликации, но и степенями принадлежности входного наблюдения и множеств, задающих правило.

Листинг кода

```
import re
from pathlib import Path


NAME_PATTERN = r"[A-Za-z][A-Za-z0-9]?"
FACT_PATTERN = re.compile(
    rf"^[ \t]*({NAME_PATTERN})[ \t]*=[ \t]*\{{([^}}\r\n]+)\}}[ \t]*\r?$",
    re.MULTILINE,
)
PAIR_PATTERN = re.compile(r"<([^,>]+),\s*((?:0|1(?:\.0+)?|0?\.\d+))>")
RULE_PATTERN = re.compile(
    rf"^[ \t]*({NAME_PATTERN})(?:\([^()\r\n]*\))?[ \t]*~>[ \t]*"
    rf"({NAME_PATTERN})(?:\([^()\r\n]*\))?[ \t]*\r?$",
    re.MULTILINE,
)


def build_godel_relation(antecedent, consequent):
    """Построить R(x, y) = I_G(A(x), B(y))."""
    relation = []
    for x, a_degree in antecedent:
        for y, b_degree in consequent:
            implication_degree = b_degree if a_degree > b_degree else 1.0
            relation.append((x, y, implication_degree))
    return relation


def max_min_composition(observation, relation):
    """Вычислить B'(y) = max_x min(A'(x), R(x, y))."""
    output_degrees = {}
    for x, observed_degree in observation:
        for source_x, y, relation_degree in relation:
            if x != source_x:
                continue
            candidate_degree = min(observed_degree, relation_degree)
            if y not in output_degrees or candidate_degree >
            ↪  output_degrees[y]:
                output_degrees[y] = candidate_degree
    return list(output_degrees.items())
```

```
def apply_fuzzy_rule(antecedent, consequent, observation):
    """Один шаг обобщённого modus ponens."""
    relation = build_godel_relation(antecedent, consequent)
    return max_min_composition(observation, relation)


def find_compatible_facts(antecedent, facts):
    """Факты, определённые на том же наборе элементов, что и посылка."""
    antecedent_domain = {element for element, _ in antecedent}
    compatible = {}
    for fact_name, memberships in facts.items():
        fact_domain = {element for element, _ in memberships}
        if fact_domain == antecedent_domain:
            compatible[fact_name] = memberships
    return compatible


def format_fuzzy_set(memberships):
    """Сохранить первоначальный формат печати результата."""
    return "{" + ",".join(f"<{element}, {degree}>" for element, degree in
    ↪  memberships) + "}"


def forward_fuzzy_inference(rules, facts):
    """Применять правила, пока база фактов не перестанет расширяться."""
    derivation_lines = []
    already_reported = []

    while True:
        facts_before_pass = len(facts)
        for antecedent_name, consequent_name in rules:
            antecedent = facts[antecedent_name]
            consequent = facts[consequent_name]
            suitable_observations = find_compatible_facts(antecedent, facts)

            for observed_name, observation in suitable_observations.items():
                derived_fact = apply_fuzzy_rule(antecedent, consequent,
                ↪  observation)
                derived_name = f"_{len(derivation_lines) + 1}"
                line = (
```

```
                    f"{{{observed_name},
                    ↪ {antecedent_name}(x)~>{consequent_name}(y)}} "
                    f"|~ {derived_name} = {format_fuzzy_set(derived_fact)}"
                )

                # Повтор уже известного исходного факта показываем один раз.
                if derived_fact in facts.values():
                    if derived_fact not in already_reported:
                        derivation_lines.append(line)
                        already_reported.append(derived_fact)
                    continue

                derivation_lines.append(line)
                facts[derived_name] = derived_fact
                already_reported.append(derived_fact)

        if len(facts) == facts_before_pass:
            break

    return "\n".join(derivation_lines)


def read_knowledge_base(filename):
    """Прочитать факты и правила из текстовой базы знаний."""
    content = Path(filename).read_text(encoding="utf-8")
    facts = {}
    rules = []

    for match in FACT_PATTERN.finditer(content):
        fact_name = match.group(1)
        body = match.group(2)
        elements = PAIR_PATTERN.findall(body)
        if body.count("<") != len(elements):
            raise ValueError(
                f"Степени принадлежности в факте {fact_name} "
                f"должны быть в диапазоне [0.0, 1.0]:
                ↪ {match.group(0).strip()}"
            )
        facts[fact_name] = [(item.strip(), float(degree)) for item, degree in
        ↪ elements]

    for match in RULE_PATTERN.finditer(content):
```

```
        rules.append((match.group(1), match.group(2)))

    return facts, rules


def main():
    facts, rules = read_knowledge_base(Path(__file__).with_name("input.txt"))
    print(forward_fuzzy_inference(rules, facts))


if __name__ == "__main__":
    main()
```

Использованные источники

1. Логические основы интеллектуальных систем. Практикум : учебно-методическое пособие / В. В. Голенков, В. П. Ивашенко, Д. Г. Колб, К. А. Уваров. — Минск : БГУИР, 2011. — 70 с. — ISBN 978-985-488-487-5. — Электронная версия: `https://libeldoc.bsuir.by/bitstream/123456789/992/2/Log_Osn.pdf` (дата обращения: 15.10.2026).

2. Python Software Foundation. `re` — Regular expression operations [Электронный ресурс] // Python 3 Documentation. — URL: `https://docs.python.org/3/library/re.html` (дата обращения: 15.10.2026).

3. Python Software Foundation. `pathlib` — Object-oriented filesystem paths [Электронный ресурс] // Python 3 Documentation. — URL: `https://docs.python.org/3/library/pathlib.html` (дата обращения: 15.10.2026).